

Compute all pairwise vector similarities within a sparse matrix (Python)

Nov 7, 2015

When we deal with some applications such as **Collaborative Filtering** (CF), computation of vector similarities may become a challenge in terms of implementation or computational performance.

Consider a matrix whose **rows** and **columns** represent **user_id** and **item_id**. A cell contains boolean or numerical value which represents the user-item information such as purchase history or item rating.

```
\item_id  1  2  3  4  ...
user_id   -----
  1  |  1  1  0  0
  2  |  0  1  0  0
  3  |  1  0  1  0
  4  |  0  0  0  0
  5  |  0  0  1  0
  ...
```

Matrix: Whether a user purchased an item or not

In a general situation, the matrix is sparse. So we may use `scipy.sparse` library to treat the matrix. On the Item-based CF, similarities to be calculated are all N^2 combinations of two items (N columns).

This post will show the efficient implementation of similarity computation with two major similarities, **Cosine similarity** and **Jaccard similarity**.

Cosine Similarity

Cosine similarity is defined as

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}.$$

Below code calculates cosine similarities between all pairwise column vectors.

Assume that the type of `mat` is `scipy.sparse.csc_matrix`.

```
import sklearn.preprocessing as pp

def cosine_similarities(mat):
    col_normed_mat = pp.normalize(mat.tocsc(), axis=0)
    return col_normed_mat.T * col_normed_mat
```

Vectors are normalized at first. And then, cosine values are determined by matrix product.

```
In [1]: import scipy.sparse as sp

In [2]: mat = sp.rand(5, 4, 0.2, format='csc') # generate random sparse matrix
[[ 0.          0.          0.          0.         ]
 [ 0.          0.          0.          0.         ]
 [ 0.          0.01970055  0.87107041  0.         ]
 [ 0.          0.67868903  0.          0.         ]
 [ 0.          0.81698843  0.          0.         ]]

In [3]: cosine_similarities(mat)
[[ 0.          0.          0.          0.         ]
 [ 0.          1.          0.01854522  0.         ]
 [ 0.          0.01854522  1.          0.         ]
 [ 0.          0.          0.          0.         ]]
```

Jaccard Similarity

The similarity is defined as

$$\frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{a} \cdot \mathbf{a} + \mathbf{b} \cdot \mathbf{b} - \mathbf{a} \cdot \mathbf{b}}.$$

Assume that the `mat` is binary (0 or 1) matrix and the type is `scipy.sparse.csc_matrix`.

```

import numpy as np

def jaccard_similarities(mat):
    cols_sum = mat.getnnz(axis=0)
    ab = mat.T * mat

    # for rows
    aa = np.repeat(cols_sum, ab.getnnz(axis=0))
    # for columns
    bb = cols_sum[ab.indices]

    similarities = ab.copy()
    similarities.data /= (aa + bb - ab.data)

    return similarities

```

Jaccard implementation is a little bit complicated. In order to eliminate python-loops, the code manipulates `scipy.sparse.csc_matrix`'s raw representation directly.

```

In [4]: mat = sp.rand(5, 4, 0.2, format='csc')
In [5]: mat.data[:] = 1 # binarize
[[ 1.  1.  0.  0.]
 [ 0.  1.  0.  0.]
 [ 1.  0.  1.  0.]
 [ 0.  0.  0.  0.]
 [ 0.  0.  1.  0.]]

In [6]: jaccard_similarities(mat)
[[ 1.          0.33333333  0.33333333  0.          ]
 [ 0.33333333  1.          0.          0.          ]
 [ 0.33333333  0.          1.          0.          ]
 [ 0.          0.          0.          0.          ]]

```

Performance

Those two implementations contain no python-loops. So the main calculation runs in a native code of numpy or scipy.

Even if the matrix has a million elements, it will be finished in a second on my laptop.

```

In [7]: mat = sp.rand(10**5, 10**4, 0.001, format='csc')
<100000x10000 sparse matrix of type '<class 'numpy.float64'>'

```

```
with 1000000 stored elements in Compressed Sparse Column format>
```

```
In [8]: %time cosine_similarities(mat)
```

```
CPU times: user 535 ms, sys: 91.1 ms, total: 626 ms
```

```
Wall time: 647 ms
```

```
In [9]: mat.data[:] = 1 # binarize
```

```
In [10]: %time jaccard_similarities(mat)
```

```
CPU times: user 857 ms, sys: 203 ms, total: 1.06 s
```

```
Wall time: 1.14 s
```

ツイート

0

コメント1件

 ログイン ▾



コメントを投稿する...

下記でログインして投稿

またはDISQUSに登録 

名前

評価順に並び替え ▾



Vyom Vats • 5年前

amazing!

^ | ▾ • 返信 • 共有 ▾

 更新を受取る  Privacy  Do Not Sell My Data

Programming/Math

algorithm, programming,
scientific computing

 [na-o-ys](#)
 [na_o_ys](#)

Programming/Math
naoyoshi0511@gmail.com

subscribe [via RSS](#)